Musterlösung: Übung 2 Linux-Rechteverwaltung

Alle Befehle und Ergebnisse wurden auf einem Ubuntu-System mit demselben Aufbau (Benutzer, Gruppen, Verzeichnisse, umask) praktisch getestet. Zu jeder Aufgabe folgt nach der Lösung ein Hintergrundabschnitt.

Teil 1: Anmelden und Orientieren

Aufgabe 1

a) Ja, srv01 ist erreichbar: 4 Pakete gesendet, 4 empfangen, 0 % Verlust. Der Name wird aufgelöst: aus `srv01` wurde `srv01.netzwerk.lan [192.168.10.5]`. IP-Adresse: **192.168.10.5**. `TTL=64` deutet auf ein Linux-System hin (Windows startet meist mit 128).

b) `ssh tim@srv01` und `ssh tim@192.168.10.5`

c) `tim` = angemeldeter Benutzer, `srv01` = Rechnername, `~` = aktuelles Verzeichnis (hier das Home-Verzeichnis), `$` = normaler Benutzer (bei root steht `#`).

d) `man chown` (Handbuchseite) und `chown --help` (Kurzhilfe).

e) `ls -la` (gleichwertig `ls -l -a` oder `ls -al`)

> **Hintergrund:** `ping` sendet Echo-Anfragen (ICMP) und misst die Antwortzeit. Der Name srv01 wird über DNS in die IP-Adresse aufgelöst. Die TTL (Time to Live) sinkt bei jedem Router um 1. Weil Client und Server im selben Netz stehen, bleibt der Startwert des Servers sichtbar: 64 bei Linux, 128 bei Windows.
>
> `ssh benutzer@rechner` baut eine verschlüsselte Verbindung (Port 22) auf. Beim ersten Mal fragt SSH nach dem Fingerabdruck des Servers, danach nach dem Passwort. Der Prompt hat die Form `benutzer@rechner:verzeichnis$`.
>
> `man` öffnet die ausführliche Anleitung (Beenden mit `q`), `--help` zeigt eine Kurzfassung. Dateien, deren Name mit einem Punkt beginnt, sind versteckt und erscheinen erst mit `-a`. Mehrere Optionen darf man zusammenschreiben (`-la`).

Teil 2: Benutzer und Gruppen

Aufgabe 2

a) UID 1004, GID 1004. Primärgruppe: `tim`. Weitere Gruppe: `sudo`.

b) tim darf Befehle mit vorangestelltem `sudo` mit Administratorrechten (root) ausführen.

c) carlas Primärgruppe ist `carla`, ihre weitere Gruppe `extern`. Gruppen von ben: `id ben`.

d) Die Primärgruppe ist die Standardgruppe eines Benutzers (`gid`). Neu angelegte Dateien erhalten sie als Gruppe. Weitere Gruppen sind zusätzliche Mitgliedschaften, sie geben nur zusätzliche Zugriffsrechte.

e) `projekt`: anna, ben. `extern`: carla.

> **Hintergrund:** Benutzer stehen in `/etc/passwd`, Gruppen in `/etc/group`. In der Tabelle der Ausgangslage steht die Primärgruppe zuerst. `id` zeigt die Gruppen, wie sie der aktuellen Sitzung zugeordnet sind.

Aufgabe 3

a) `sudo adduser tim projekt` (gleichwertig `sudo usermod -aG projekt tim` oder `sudo addgroup tim projekt`)

b) Die Gruppenverwaltung ändert Systemdateien (`/etc/group`). Das darf nur root. tim ist in der Gruppe `sudo` und kann deshalb `sudo` verwenden.

c) Gruppenmitgliedschaften werden beim Anmelden gelesen und der Sitzung mitgegeben. Die laufende Shell kennt noch die alte Liste. tim muss sich abmelden (`exit`) und neu mit `ssh tim@srv01` anmelden.

d) `id` zeigt dann `Gruppen=1004(tim),27(sudo),1010(projekt).`

> **Hintergrund:** Die Änderung steht sofort in `/etc/group`, aber bereits laufende Prozesse behalten ihre Gruppenliste. `adduser benutzer gruppe` mit zwei Namen ist eine Debian/Ubuntu-Schreibweise. Ab hier gilt: tim ist Mitglied in `projekt`.

Aufgabe 4

a) `sudo usermod -aG extern ben`

b) `-a` bedeutet anhängen (append), `-G` nennt die zusätzlichen Gruppen. Ohne `-a` würde die gesamte Liste ersetzt: ben wäre danach nur noch in `extern` und nicht mehr in `projekt` und würde seinen Zugriff auf die Projektdaten verlieren.

c) `sudo` ist hier überflüssig, denn `id ben` braucht keine besonderen Rechte. Gründe gegen unnötiges `sudo`: (1) Fehler und Tippfehler wirken dann mit Administratorrechten und können Schaden anrichten. (2) Dateien, die mit `sudo` angelegt werden, gehören root und verursachen später Rechteprobleme.

> **Hintergrund:** Prinzip der minimalen Rechte (Least Privilege): Jeder Benutzer und jeder Befehl bekommt nur so viele Rechte wie nötig. Auch das Protokoll in `/var/log/auth.log` wird übersichtlicher, wenn `sudo` selten ist.

Teil 3: Rechte lesen

Aufgabe 5

Eintrag	Typ	Besitzer	Gruppe	Größe	Datum	Rechte Besitzer	Rechte Gruppe	Rechte others
`archiv`	Verzeichnis	anna	projekt	4096	Nov 2 16:48	`rwx`	`rwx`	`---`
`konzept.txt`	Datei	anna	anna	2048	Nov 3 10:15	`rw-`	`r--`	`r--`

> **Hintergrund:** Eine Zeile von `ls -l` lautet: Typ (`-` Datei, `d` Verzeichnis, `l` Link), dreimal drei Rechte (Besitzer, Gruppe, others), Anzahl der Links (hier nicht gefragt), Besitzer, Gruppe, Größe in Byte, Datum der letzten Änderung, Name. Bei Verzeichnissen steht meist 4096, das ist die Größe des Verzeichniseintrags, nicht des Inhalts.

Aufgabe 6

Recht	Bei einer Datei	Bei einem Verzeichnis
`r`	Inhalt lesen	Inhalt auflisten (Namen der Einträge sehen)
`w`	Inhalt ändern	Einträge anlegen, löschen und umbenennen (nur zusammen mit `x`)
`x`	als Programm oder Skript ausführen	in das Verzeichnis wechseln und auf die Einträge zugreifen

> **Hintergrund:** Ein Verzeichnis ist eine Liste von Namen. `r` erlaubt, die Liste zu lesen, `x` erlaubt, durch das Verzeichnis „hindurchzugehen“. Das Löschen einer Datei hängt deshalb von den Rechten des Verzeichnisses (`w` und `x`) ab, nicht von den Rechten der Datei.

Aufgabe 7

symbolisch	oktal	oktal	symbolisch
`rwxr-x---`	`750`	`700`	`rwx------`
`rw-r-----`	`640`	`644`	`rw-r--r--`
`rwxrwxr-x`	`775`	`770`	`rwxrwx---`

> **Hintergrund:** Jede Rechtegruppe ist eine Summe: `r` = 4, `w` = 2, `x` = 1. Die drei Ziffern gelten der Reihe nach für Besitzer, Gruppe und others. Beispiel `rwxr-x---`: 4+2+1 = 7, 4+0+1 = 5, 0 = 750.

Aufgabe 8

Person	gilt	Rechte	darf schreiben?
anna	Besitzer	`r--`	nein
ben	Gruppe	`rw-`	ja
carla	others	`---`	nein
tim	Gruppe (seit Aufgabe 3 in `projekt`)	`rw-`	ja

Anna ist Besitzerin. Die Klasse „Besitzer“ wird zuerst geprüft, und der erste Treffer entscheidet. Dass die Gruppe `rw-` hätte, spielt für anna keine Rolle.

> **Hintergrund:** Linux prüft in der Reihenfolge Besitzer, Gruppe, others und wendet nur die erste passende Klasse an. Die Rechte werden nie zusammengezählt. Deshalb kann ein Besitzer weniger Rechte haben als die Gruppe seiner eigenen Datei.

Aufgabe 9

```
/srv/projekt        root : projekt    750
├── doku            anna : projekt    770
├── code            ben : projekt     770
│   └── build       ben : projekt     755
└── austausch       anna : extern     770
```

> **Hintergrund:** `ls -ld verzeichnis` zeigt das Verzeichnis selbst, `ls -l verzeichnis` seinen Inhalt. Die Rechte `drwxrwx---` ergeben 770, `drwxr-xr-x` ergeben 755. Die Zahl nach den Rechten ist die Linkzahl: 2 plus die Anzahl der Unterverzeichnisse, daher 5 bei `/srv/projekt` (drei Unterverzeichnisse).

Teil 4: Dateien und Verzeichnisse verwalten

Aufgabe 10

a) `cd /srv/projekt/doku`

b) `tim@srv01:/srv/projekt/doku$`

c) `cd ../code`

d) `cd ~` (oder nur `cd`)

e) `cd ../../srv/projekt/doku` (`..` = `/home`, `../..` = `/`)

f) `.` = aktuelles Verzeichnis, `..` = übergeordnetes Verzeichnis, `~` = Home-Verzeichnis des angemeldeten Benutzers.

> **Hintergrund:** Absolute Pfade beginnen mit `/` und gelten von überall. Relative Pfade beginnen im aktuellen Verzeichnis. Dass tim überall hinkommt, liegt an seiner Mitgliedschaft in `projekt` (Aufgabe 3): `/srv/projekt` hat für die Gruppe `r-x`. `cd -` springt zusätzlich zum zuletzt besuchten Verzeichnis zurück.

Aufgabe 11

a) `mkdir code/testlauf` (gleichwertig `mkdir ./code/testlauf`)

b) `code` gehört ben:projekt mit 770. tim ist Mitglied von `projekt`, die Gruppe hat `rwx`, also darf tim dort anlegen.

c) `ls -l code`

d) `drwxr-x--- 2 tim tim 4096 Nov 10 09:20 testlauf`

> **Hintergrund:** Zum Anlegen braucht man `w` und `x` im Elternverzeichnis. Neue Verzeichnisse starten mit 777, die umask 0027 entzieht 027, daraus wird 750. Die Gruppe ist tims Primärgruppe `tim`, weil kein SGID-Bit gesetzt ist.

Aufgabe 12

Eintrag	Rechte symbolisch	Rechte oktal	Besitzer	Gruppe
`ideen.txt`	`rw-r-----`	640	tim	tim
`skizzen`	`rwxr-x---`	750	tim	tim

b) Besitzer 6 − 0 = 6, Gruppe 6 − 2 = 4, others 6 − 7 → 0. Ergebnis 640. (Verzeichnis: 777 − 027 = 750.)

c) anna (umask 0022): Datei 644, Verzeichnis 755.

> **Hintergrund:** Die umask legt fest, welche Rechte neuen Einträgen entzogen werden. Dateien starten mit 666, weil neue Dateien nie automatisch ausführbar sein sollen, Verzeichnisse mit 777. Genau genommen werden die Bits der umask gelöscht, es ist keine normale Subtraktion. Die aktuelle umask zeigt der Befehl `umask`.

Aufgabe 13

a) `cp bericht.txt /srv/projekt/doku/`

b) `mv notiz.txt /srv/projekt/code/`

c) `cp notiz.txt /srv/projekt/code/` und danach `rm notiz.txt`

d) Nach a) liegt `bericht.txt` noch im Home-Verzeichnis (Kopie). Nach b) ist `notiz.txt` nicht mehr im Home-Verzeichnis.

> **Hintergrund:** `cp` erzeugt eine neue Datei: Besitzer ist der ausführende Benutzer, die Rechte ergeben sich aus der Quelle und der umask, der Zeitstempel ist neu. `mv` ändert innerhalb eines Dateisystems nur den Verzeichniseintrag, Besitzer, Rechte und Zeitstempel bleiben erhalten. `mv` über Dateisystemgrenzen kopiert und löscht. `rm` löscht endgültig, es gibt keinen Papierkorb. Ordner brauchen `cp -r` und `rm -r`. `sudo` ist nicht nötig, weil `doku` und `code` der Gruppe `projekt` gehören und `rwx` erlauben.

Teil 5: Rechte und Eigentümer ändern

Aufgabe 14

a) `chmod 700 skizzen`

b) `chmod 660 ideen.txt`

c) a) `rwx------`, b) `rw-rw----`

> **Hintergrund:** Die oktale Schreibweise setzt alle neun Rechtebits auf einmal. `chmod` darf nur der Besitzer (oder root). tim ist Besitzer, deshalb kein `sudo`.

Aufgabe 15

a) `chmod g+w bericht.txt`

b) `chmod o-rwx weitergabe.txt`

c) `chmod u=rwx,g=,o= material` (gleichwertig `chmod 700 material`)

> **Hintergrund:** Symbolisch: Klasse `u` (Besitzer), `g` (Gruppe), `o` (others) oder `a` (alle), dann `+` (hinzufügen), `-` (entfernen) oder `=` (genau setzen), dann `r`, `w`, `x` und bei Verzeichnissen `s`. Bei `=` entfallen alle nicht genannten Rechte der Klasse. Mehrere Angaben trennt man mit Komma. Die symbolische Schreibweise ändert gezielt einzelne Rechte, die oktale setzt alles neu.

Aufgabe 16

Nr.	Was muss geändert werden?	Befehl	sudo nötig?
(1)	Rechte (die Gruppe `projekt` ist richtig, `g` hat nur `r--`)	`chmod g+w entwurf.txt`	nein, tim ist Besitzer
(2)	Gruppe (Rechte `rw-rw----` stimmen schon)	`sudo chgrp projekt /srv/projekt/doku/angebot.txt`	ja, tim ist nicht Besitzer
(3)	Besitzer	`sudo chown anna /srv/projekt/code/export.csv`	ja, nur root darf den Besitzer ändern

> **Hintergrund:** Zuerst klären, was nicht stimmt: Rechte, Gruppe oder Besitzer? Wer darf was? `chmod`: der Besitzer (oder root). `chgrp`: der Besitzer, aber nur auf Gruppen, in denen er selbst Mitglied ist (oder root). `chown` auf einen anderen Besitzer: nur root. Gleichwertig zu `chgrp projekt datei` ist `chown :projekt datei`.

Aufgabe 17

a) ben ist weder Besitzer der Datei noch Mitglied der Gruppe `tim`. Für ihn gilt deshalb die Klasse others mit `---`. Der Weg zur Datei ist frei, ben ist ja Mitglied von `projekt`.

b) `chgrp projekt /srv/projekt/doku/protokoll.txt`

c) `chown :projekt /srv/projekt/doku/protokoll.txt` (auch `chown tim:projekt …`)

d) `chmod o+r` öffnet die Datei für alle Benutzer des Servers und widerspricht dem Prinzip, others nur ausdrücklich zu öffnen. ben in die Gruppe `tim` aufzunehmen, gäbe ihm Zugriff auf alles, was der Gruppe `tim` gehört, und erfordert `sudo`.

> **Hintergrund:** Nach `chgrp` steht `-rw-r----- tim projekt`. Die Gruppe darf lesen, das genügt ben. Soll ben auch schreiben, braucht es zusätzlich `chmod g+w`. Kein `sudo` nötig: tim ist Besitzer und selbst Mitglied von `projekt`. Ein normaler Benutzer darf die Gruppe seiner Dateien nur auf Gruppen ändern, in denen er Mitglied ist.

Aufgabe 18

a) Reihenfolge mit nur einem `sudo`:

```
tim@srv01:/srv/projekt/doku$ chmod 660 plan.txt
tim@srv01:/srv/projekt/doku$ sudo chown anna:projekt plan.txt
```

b) `chmod` darf nur der Besitzer. Solange tim noch Besitzer ist, braucht er dafür kein `sudo`. Den Besitzer auf einen anderen Benutzer zu ändern darf nur root, dafür ist `sudo` nötig. Wird zuerst `chown` ausgeführt, gehört die Datei anna. Ein `chmod 660 plan.txt` ohne `sudo` scheitert dann mit der Meldung „Die Operation ist nicht erlaubt“ (Operation not permitted). Mit `sudo` ginge es, bräuchte aber zwei `sudo`-Aufrufe.

> **Hintergrund:** Merkregel: Erst `chmod` und `chgrp`, solange man Besitzer ist, zuletzt `chown` auf einen anderen Besitzer. `chown anna:projekt` setzt Besitzer und Gruppe in einem Schritt, `chown :projekt` nur die Gruppe, `chown anna` nur den Besitzer.

Teil 6: Das SGID-Bit

Aufgabe 19

a) `ben` (seiner Primärgruppe).

b) Das SGID-Bit (Set Group ID) am Verzeichnis `doku`. Neue Dateien und Unterverzeichnisse darin erben die Gruppe des Verzeichnisses (`projekt`) statt der Primärgruppe des Erstellers.

c) `sudo chmod g+s /srv/projekt/doku`

d) anna, denn sie ist Besitzerin von `doku`. Nur der Besitzer (oder root) darf `chmod` ausführen. tim ist nicht Besitzer und braucht deshalb `sudo`; ohne `sudo` kommt „Operation not permitted“.

> **Hintergrund:** Spezialrechte sind SUID (4), SGID (2) und Sticky-Bit (1). Auf einem Verzeichnis bewirkt SGID, dass die Gruppe weitervererbt wird. Unterverzeichnisse erben zusätzlich das SGID-Bit, damit die Wirkung im ganzen Baum erhalten bleibt. Typischer Einsatz sind gemeinsame Projektverzeichnisse.

Aufgabe 20

Zeitpunkt	symbolisch	oktal
vorher	`drwxrwx---`	770
nachher	`drwxrws---`	2770

b) Das SGID-Bit wird an der Stelle des Gruppen-`x` angezeigt. Weil dort vorher ein `x` gesetzt war, erscheint ein kleines `s`.

> **Hintergrund:** Die führende Ziffer 2 in der oktalen Schreibweise steht für SGID. Wäre für die Gruppe kein `x` gesetzt, würde ein großes `S` erscheinen (SGID ohne Ausführungsrecht).

Aufgabe 21

Eintrag	Besitzer	Gruppe	Rechte (symbolisch)
`neu.txt`	ben	projekt	`rw-r--r--`
`unterordner`	ben	projekt	`rwxr-sr-x`
`konzept.txt`	anna	anna	`rw-r--r--`

b) Das SGID-Bit wirkt nur auf Einträge, die nach dem Setzen angelegt werden. Vorhandene Dateien bleiben unverändert (nicht rückwirkend).

c) `sudo chgrp projekt /srv/projekt/doku/konzept.txt` (`sudo`, weil tim nicht Besitzer ist).

> **Hintergrund:** `neu.txt`: 666 − 022 = 644. Der Ordner: 777 − 022 = 755, dazu kommt das geerbte SGID-Bit, daher `rwxr-sr-x`. Der Besitzer ist immer der Ersteller, SGID betrifft nur die Gruppe.

Teil 7: Zugriff analysieren

Aufgabe 22

Aktion	anna	ben	carla	tim
a) Inhalt von `/srv/projekt/doku` auflisten	ja	ja	nein	ja
b) in `/srv/projekt/code/build` wechseln	ja	ja	nein	ja
c) in `/srv/projekt/code/build` eine Datei anlegen	nein	ja	nein	nein
d) Inhalt von `/srv/projekt/austausch` auflisten	ja	ja	nein	nein
e) `konzept.txt` nach `austausch` kopieren	ja	ja	nein	nein

Begründungen: carla (a): `/srv/projekt` hat für others `---`, ohne `x` darauf kommt sie nicht zu `doku`. tim (d): `austausch` gehört anna:extern mit 770. tim ist weder Besitzer noch in `extern`, für ihn gilt others `---`. anna (c): `build` gehört ben:projekt mit 755. Für anna gilt die Gruppe `projekt` mit `r-x`, es fehlt `w` zum Anlegen.

> **Hintergrund:** Zeile e): Zum Kopieren braucht man `r` auf der Quelldatei, `x` auf dem ganzen Weg dorthin sowie `w` und `x` im Zielverzeichnis. tim kann `konzept.txt` lesen (others `r--`), scheitert aber am Ziel `austausch`. Wechseln braucht `x`, Auflisten `r`, Anlegen `w` und `x` im Verzeichnis. Dazu kommt immer das `x` auf allen übergeordneten Verzeichnissen.

Aufgabe 23

1. Richtig: ben kann in `/srv/projekt/doku` Dateien anlegen. Die Gruppe `projekt` hat `rwx` (2770), ben ist Mitglied, der Weg ist frei (`r-x` auf `/srv/projekt`).
2. Richtig: carla kann `/srv/projekt/austausch` nicht auflisten. Ihr fehlt `x` auf `/srv/projekt`. Die Mitgliedschaft in `extern` nützt nichts, wenn der Weg versperrt ist.
3. Richtig: tim kann in `/srv/projekt/code/build` wechseln. Als Mitglied von `projekt` hat er dort `r-x` (755), und `x` genügt zum Wechseln.
4. Richtig: anna kann in `/srv/projekt/code/build` keine Dateien anlegen. Die Gruppe `projekt` hat dort nur `r-x`, es fehlt `w`.

> **Hintergrund:** Bei Aussagen mit Verneinung zuerst festhalten, was genau behauptet wird („kann“ oder „kann nicht“), und erst dann prüfen. „Kann keine Dateien anlegen“ bedeutet: `w` oder `x` fehlt irgendwo im Weg oder im Zielverzeichnis.

Aufgabe 24

a) carla fällt bei `/srv/projekt` (root:projekt, 750) in die Klasse others mit `---`. Ohne `x` auf einem Verzeichnis im Pfad erreicht sie `austausch` nicht, auch wenn sie dort als Mitglied von `extern` `rwx` hätte.

b) Als Mitglied von `projekt` hätte carla Zugriff auf `doku` und `code`, also auf alle Projektdaten. Das ist viel mehr als nötig und widerspricht dem Prinzip der minimalen Rechte.

c) `sudo chmod o+x /srv/projekt` (gleichwertig `sudo chmod 751 /srv/projekt`)

d) Zustand nach dem Befehl:

Aktion	ja/nein	Begründung
(1) `/srv/projekt/austausch` auflisten	ja	`x` für others auf `/srv/projekt`, auf `austausch` hat `extern` (carla) `rwx`
(2) `/srv/projekt` auflisten	nein	others haben nur `--x`, kein `r`
(3) in `/srv/projekt/doku` wechseln	nein	others haben auf `doku` `---`
(4) in `/srv/projekt/code` wechseln	nein	others haben auf `code` `---`

Hintergrund: `x` ohne `r` auf einem Verzeichnis bedeutet: durchqueren erlaubt, auflisten nicht. Die Änderung ist die sparsamste Lösung, weil sie nur an einer Stelle ein einziges Recht ergänzt und kein Projektverzeichnis öffnet. Danach gilt `/srv/projekt` mit `drwxr-x--x` (751) als neue Ausgangslage.

Aufgabe 25

1. **Klasse bestimmen:** Für jedes Verzeichnis und jede Datei im Pfad prüfen, ob die Person Besitzer ist, sonst Mitglied der Gruppe (Primärgruppe oder weitere), sonst others. Der erste Treffer gilt.
2. **Rechte der Klasse mit dem Bedarf vergleichen:** Wechseln braucht `x`, Auflisten `r`, Lesen einer Datei `r`, Anlegen und Löschen `w` und `x` im Verzeichnis. Kopieren braucht `r` an der Quelle und `w` und `x` am Ziel.
3. **Den ganzen Pfad prüfen:** `x` muss auf jedem übergeordneten Verzeichnis ab `/` vorhanden sein. Eine einzige fehlende Stelle genügt, damit der Zugriff scheitert.

Hintergrund: Bei einem Zugriffsproblem hilft `ls -ld` für jeden Teil des Pfades. Die sparsame Lösung ändert genau die Stelle, an der etwas fehlt (Gruppe, Besitzer oder ein einzelnes Recht), und nicht pauschal `777` oder `o+rwx`.

Abdeckung der Kannliste

Kannliste-Punkt	Aufgabe
Anmelden und Orientieren	
per `ssh benutzer@rechner` einloggen	1b
mit `ping` Erreichbarkeit und Namensauflösung prüfen	1a
Terminalausgabe lesen und Informationen ableiten	1a, 5, 9
Prompt lesen und bei Pfadwechsel anpassen	1c, 10b
`man` und `--help` nutzen	1d, 1e
Benutzer und Gruppen	
`id` und `id benutzer` auslesen	2
Primärgruppe und weitere Gruppen unterscheiden	2d
Benutzer zu Gruppe hinzufügen (`adduser`, `usermod -aG`)	3a, 4a, 4b
erklären, warum man sich neu einloggen muss	3c
Benutzer- und Gruppenlisten auswerten	2e, 22
wann `sudo` nötig ist, warum nicht unnötig	3b, 4c, 16, 18
Rechte lesen	
`ls -l` / `ls -la` vollständig lesen	5
`r`, `w`, `x` bei Dateien und Verzeichnissen unterscheiden und erklären	6
symbolisch und oktal umrechnen	7
bestimmen, welche Rechtegruppe für eine Person gilt	8

Kannliste-Punkt	Aufgabe
Verzeichnisbaum lesen und fehlende Angaben ergänzen	9
Rechte und Eigentümer ändern	
`chmod` oktal	14
`chmod` symbolisch (`g+s`, `o-rwx`, `u=rwx`)	15, 19c, 24c
`chown besitzer:gruppe`, `chown :gruppe`	17c, 18
`chgrp`	16, 17b
entscheiden, welche Änderung nötig ist	16
begründen, warum Rechte für others vermieden werden	17d, 24b
Reihenfolge von `chmod` und `chown`, `sudo` dafür	18
SGID-Bit	
Wirkung erklären	19b
SGID setzen	19c
Rechte oktal (`2770`) und symbolisch (`rwxrws---`) angeben	20a
erklären, warum `s` statt `x` erscheint	20b
erklären, dass SGID nicht rückwirkend wirkt	21b, 21c
begründen, wer SGID setzen darf	19d
Dateien und Verzeichnisse verwalten	
`cd` mit absoluten und relativen Pfaden	10
`.`, `..`, `~` einsetzen	10e, 10f
`mkdir` mit relativem Pfad	11a
`ls -l` für ein Verzeichnis ohne Wechsel	11c
Rechte und Gruppe neuer Dateien vorhersagen (umask, SGID)	12, 21
`mv`, `cp`, `rm`	13
`cp` mit `rm` gleich `mv`, Unterschied erklären	13c, 13d
Zugriff analysieren	
den gesamten Pfad prüfen	22, 24, 25
wechseln, auflisten, anlegen, kopieren	22a–e
Aussagen mit Verneinung auswerten	23
Änderungen als neue Ausgangslage berücksichtigen	24d
Fehlersituation analysieren, sparsame Lösung vorschlagen	17, 24